

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет
ИТМО»

Дисциплина: Информационная безопасность

Лабораторная работа № 1

Разработка защищенного REST API с интеграцией в
CI/CD

Группа: Р3308

Выполнил: Елисеев Константин
Иванович

Санкт-Петербург
2026

ОПИСАНИЕ И РЕЗУЛЬТАТЫ РАБОТЫ

Цель: разработать API с защитой от SQL-инъекций, экранированием данных и аутентификацией JWT; включить тесты и сканеры безопасности в CI.

Ниже приведено содержимое README проекта. Команды, меры защиты и результаты относятся к лабораторной № 1.

Дисциплина: «Информационная безопасность», Университет ИТМО.

Автор: Елисеев Константин Иванович, группа Р3308.

Приложение выдаёт JWT после проверки логина и пароля, позволяет читать и создавать посты. Стек: Python, Flask, SQLite, SQLAlchemy, PyJWT и bcrypt. Python выбран как знакомый язык; Flask позволяет явно показать маршруты, middleware и меры защиты в небольшом приложении.

Репозиторий (<https://github.com/virdxk/infosec-lab1>) · GitHub Actions (<https://github.com/virdxk/infosec-lab1/actions/workflows/ci.yml>) · Протокол curl (<docs/curl-session.md>)

Запуск

Нужен Python 3.12 или новее. Команды рассчитаны на macOS/Linux.

```
git clone git@github.com:virdxk/infosec-lab1.git
cd infosec-lab1
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements-dev.txt
cp .env.example .env
python -c 'import secrets; print(secrets.token_urlsafe(48))'
```

Запишите сгенерированное значение после JWT_SECRET= в .env.

Файл .env исключён из Git. Затем:

```
python seed.py
python wsgi.py
```

Сервер доступен по <http://127.0.0.1:5000>. Данные хранятся в app.db в текущем каталоге. Повторный запуск seed.py не дублирует демонстрационные записи.

Таблица 1 — Параметры и назначение

Настройка	Значение по умолчанию	Назначение
JWT_SECRET	Не задано	Секрет HS256, не менее 32 байт; генерируется случайно
JWT_TTL_SECONDS	3600	Срок действия токена в секундах
DATABASE_URL	sqlite:///app.db	Подключение к БД
BCRYPT_ROUNDS	12	Стоимость хэширования паролей
PORT	5000	Порт локального сервера

Демонстрационные учётные записи: alice / Al1ce-Str0ng-Pass! и admin / Adm1n-Str0ng-Pass!. Это учебные данные из seed.py. В БД сохраняются только bcrypt-хэши. Различий в правах между этими пользователями нет.

API и примеры вызова

Таблица 2 — Параметры и назначение

Метод	Назначение	JWT	Успех
POST /auth/login	Вход по логину и паролю	Не нужен	200
GET /api/data	Список постов	Обязателен	200
POST /api/posts	Создание поста	Обязателен	201
GET /health	Проверка доступности сервера	Не нужен	200

Вход

```
BASE_URL=http://127.0.0.1:5000
TOKEN=$(curl -fsS "$BASE_URL/auth/login" \
  -H 'Content-Type: application/json' \
  --data '{"username":"alice","password":"Al1ce-Str0ng-Pass!"}' \
  | python -c 'import json,sys; print(json.load(sys.stdin)
["access_token"])')
```

Успешный ответ содержит access_token, token_type: "Bearer" и expires_in: 3600. Токен передаётся заголовком Authorization: Bearer <token>.

Чтение и создание поста

```
curl -i "$BASE_URL/api/data" -H "Authorization: Bearer $TOKEN"
```

```
curl -i "$BASE_URL/api/posts" \  
-H "Authorization: Bearer $TOKEN" \  
-H 'Content-Type: application/json' \  
--data '{"title":"Учебный пост","body":"Пример текста"}'
```

Чтение возвращает объект с count и массивом items. Создание возвращает пост с полями id, title, body, author, created_at. Автор определяется по проверенному JWT, прислать чужой ID автора нельзя.

Проверка отказа и экранирования

```
curl -i "$BASE_URL/api/data"
```

```
curl -i "$BASE_URL/api/posts" \  
-H "Authorization: Bearer $TOKEN" \  
-H 'Content-Type: application/json' \  
--data '{"title":"XSS probe","body":"<script>alert(1)</script>"}'
```

Первый запрос получает 401. Второй получает 201, а body содержит `<script>alert(1)</script>`. При повторном чтении пост также возвращается экранированным.

Ошибки: 400 для неверного формата или отсутствующих строковых полей; 401 для неверного пароля, отсутствующего, поддельного или просроченного JWT; 404 для неизвестного адреса; 405 для неподдерживаемого метода. Произвольных лимитов длины логина и постов нет. Пароль длиннее 72 байт получает обычный отказ во входе из-за технического ограничения bcrypt.

Реализованные меры защиты

SQL Injection

В app/auth.py поиск выполняется через `select(User).where(User.username == username)`. SQLAlchemy передаёт имя связанным параметром. Строка `' OR '1'='1' --` остаётся значением имени пользователя и не меняет условие SQL. Конкатенации пользовательского ввода с SQL нет.

XSS

app/sanitize.py экранирует title, body и имя автора через markupsafe.escape. Исходный текст сохраняется в БД; экранирование применяется при создании ответа и при чтении. jsonify задаёт JSON-формат ответа. Это выполняет требование ТЗ об экранировании возвращаемого пользовательского текста; будущий браузерный интерфейс также должен безопасно вставлять эти данные.

Аутентификация и JWT

app/auth.py проверяет пароль через bcrypt.checkpw и выдаёт токен только при совпадении. Соль генерируется отдельно для каждого пароля и хранится внутри bcrypt-хэша вместе с параметрами алгоритма.

app/security.py выпускает токен HS256 с sub, username, role, iat, exp. Подпись защищает данные от изменения, но не шифрует их. Алгоритм проверки фиксирован сервером; sub, iat, exp обязательны.

В начале app/api.py зарегистрирован api_bp.before_request(authenticate_request). Этот hook вызывает функцию из app/middleware.py перед защищёнными обработчиками. Middleware проверяет заголовок, подпись, срок и существование пользователя в БД. После успеха ID пользователя записывается в g, затем Flask вызывает обработчик. При ошибке возвращается 401. Публичные /auth/login и /health не входят в этот blueprint; автоматический OPTIONS сообщает разрешённые методы без выдачи данных.

Все вошедшие пользователи читают общие посты. Поле role не реализует ролевые ограничения. В учебной работе нет MFA, ограничения попыток входа и отзыва отдельных JWT. Одинаковые сообщения при неверном пароле и неизвестном пользователе не устраняют различия во времени ответа.

Автоматические проверки

```
pytest -q
bandit -r app seed.py wsgi.py -ll
pip-audit -r requirements.txt --strict
```

.github/workflows/ci.yml запускается при push и pull request. Три независимые задачи:

1. test: установка зависимостей и pytest.
2. sast: Bandit; замечания уровня medium/high завершают проверку ошибкой, полный JSON-отчёт сохраняется как bandit-report.

3. sca: OWASP Dependency-Check 13.0.0 и дополнительный pip-audit; HTML/JSON-отчёты сохраняются как sca-reports даже при неудаче проверки.

Dependency-Check анализирует отдельный requirements.txt со всеми установленными прикладными зависимостями, включая транзитивные. Для Python включён --enableExperimental. Порог отказа --failOnCVSS 7 соответствует высоким и критическим уязвимостям. База загружается из официальных JSON 2.0 feeds NVD и кэшируется; аккаунт Snyk и NVD API key не требуются. Дополнительный pip-audit проверяет Python-пакеты по специализированной базе и завершает задачу ошибкой при любой найденной известной уязвимости.

Для этого Python-проекта отключены RetireJS (JavaScript), OSS Index (отдельный сервис с учётными данными), автоматическая загрузка исключений и проверка новой версии сканера. NVD-анализ включён. В .github/dependency-check-suppressions.xml исключено одно подтверждённое ложное совпадение: CVE-2025-45770 для PyJWT 2.14.0. В записи NVD указан PHP-пакет lcobucci/jwt, а здесь используется Python-пакет PyJWT. Исключение ограничено конкретными пакетом, версией и CVE; остальные находки проверяются как обычно. Ошибка скачивания или сканирования не превращается в успешную проверку.

При проверке 30.09.2026 аудит обнаружил известные уязвимости PyJWT 2.13.0. Версия обновлена до 2.14.0; повторный pip-audit не обнаружил известных уязвимостей. Тесты входа, подписи JWT и контроля доступа после обновления проходят.

Результаты и материалы

Проверенный запуск GitHub Actions от 30.09.2026: CI #7, коммит e312fb9 (<https://github.com/virdxk/infosec-lab1/actions/runs/36704738027>). Все три задачи завершились успешно:

- pytest: 51 тест пройден;
- Bandit: 0 замечаний;
- OWASP Dependency-Check: 13 зависимостей, 0 активных находок, одно обоснованное ложное совпадение исключено;
- pip-audit: 13 зависимостей, 0 известных уязвимостей.

Сканирование отражает состояние баз уязвимостей на момент запуска и не доказывает отсутствие всех возможных проблем.

Материалы проверки:

- Фактические запросы и ответы curl (docs/curl-session.md): 11 сценариев, отдельная временная БД.
- Отчёт Bandit из CI (docs/reports/bandit.json).
- Отчёт pip-audit из CI (docs/reports/pip-audit.json).
- Отчёт OWASP Dependency-Check из CI (docs/reports/dependency-check-report.html) и его JSON-версия (docs/reports/dependency-check-report.json).

- Запуски pipeline, завершившиеся успешно (<https://github.com/virdxk/infosec-lab1/actions/workflows/ci.yml?query=is%3Asuccess>): новые коммиты с материалами отчёта также запускают CI; скриншоты ниже относятся к проверке кода в e312fb9.

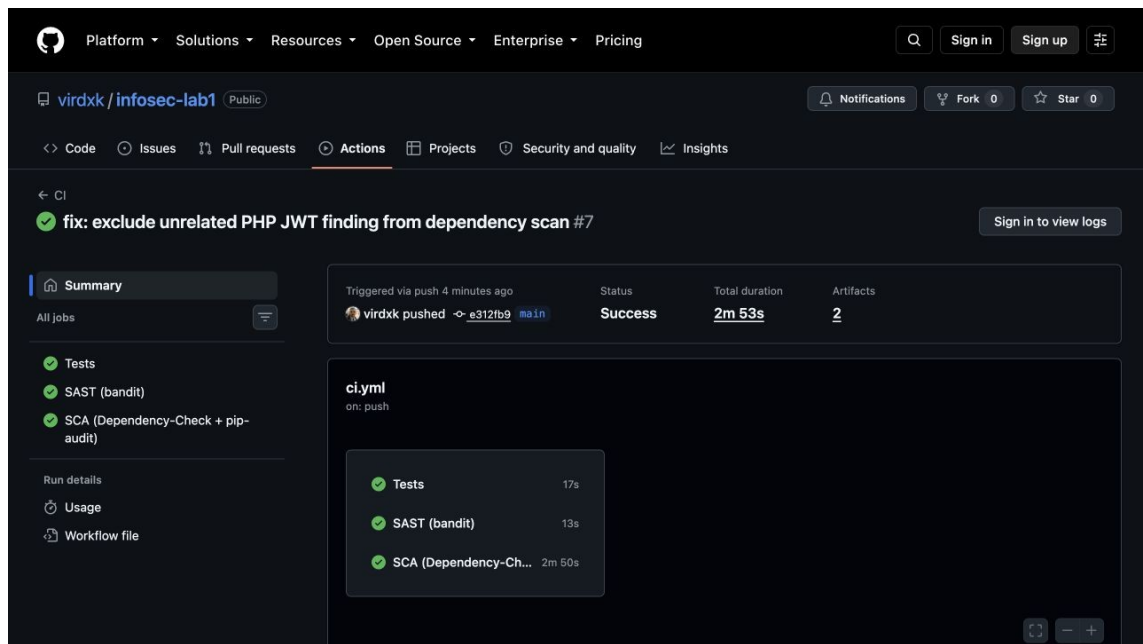


Рисунок 1 — Успешный запуск GitHub Actions

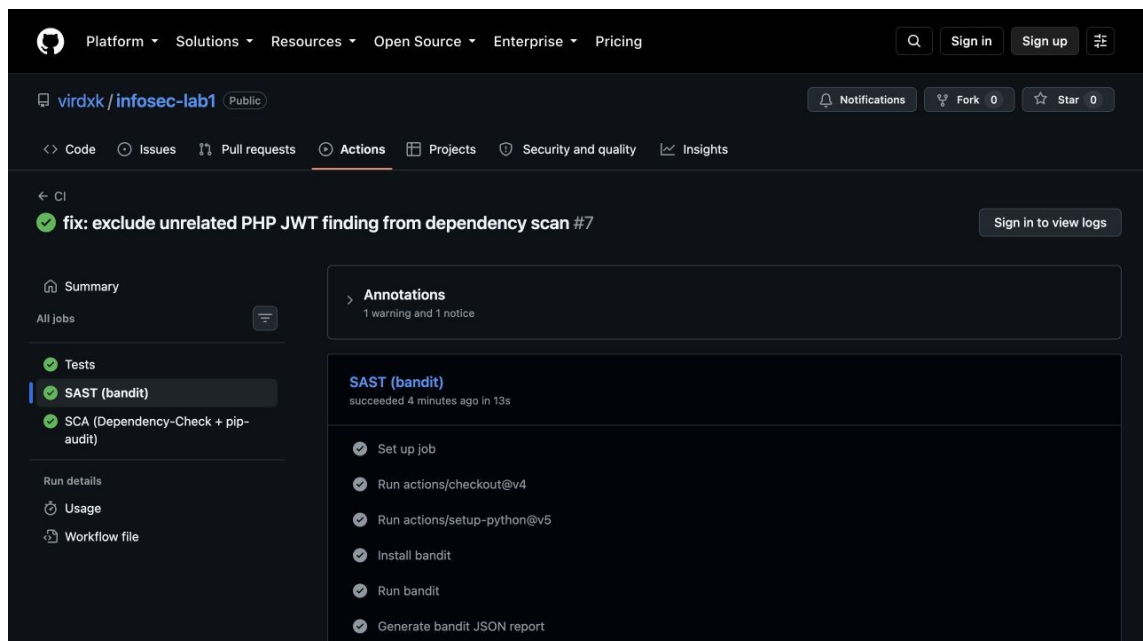


Рисунок 2 — Успешная проверка SAST: Bandit

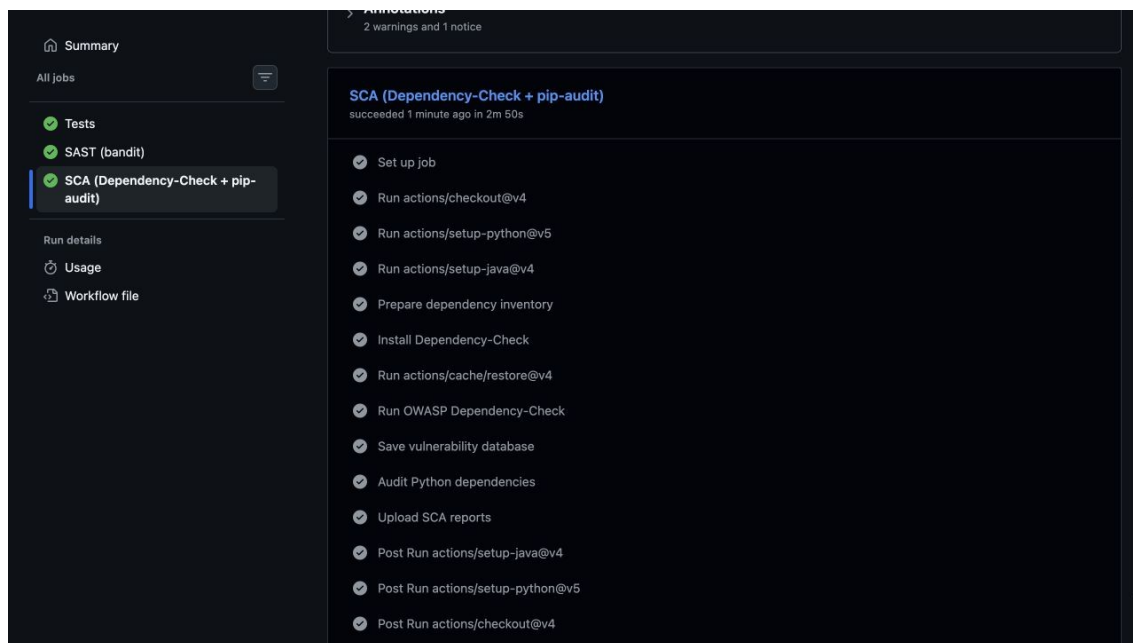


Рисунок 3 — Успешная проверка SCA: Dependency-Check и pip-audit

Навигация по коду

Таблица 3 — Параметры и назначение

Файл	Ответственность
app/__init__.py	Создание приложения, конфигурация, подключение blueprints
app/auth.py	Вход по логину и паролю
app/middleware.py	Проверка JWT перед защищёнными обработчиками
app/security.py	Выдача и проверка JWT
app/api.py	Чтение и создание постов
app/models.py	Модели User/Post, bcrypt, подключение к БД
app/sanitize.py	Экранирование данных ответа
seed.py	Демонстрационные записи

Источники

- Отчёт по лабораторной № 1 (docs/report.pdf); оригинал задания преподавателя предоставлен отдельно.
- Flask (<https://flask.palletsprojects.com/en/stable/>).

- OWASP SQL Injection Prevention
(https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html).

- OWASP Cross Site Scripting Prevention
(https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html).

- OWASP Password Storage
(https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html)

.

- JWT (<https://www.jwt.io/introduction>).

- Dependency-Check: Pip Analyzer
(<https://dependency-check.github.io/DependencyCheck/analyzers/pip.html>).

- NVD: CVE-2025-45770 (<https://nvd.nist.gov/vuln/detail/CVE-2025-45770>) и документация PHP-библиотеки lcobucci/jwt (<https://lcobucci-jwt.readthedocs.io/en/stable/>): обоснование ложного совпадения.

- Dependency-Check: исключения ложных совпадений (<https://dependency-check.github.io/DependencyCheck/general/suppression.html>).

ПРОВЕРКА HTTP-ЗАПРОСОВ

Выполнено: 2026-09-30T10:50:27.283550+00:00.

Использованы отдельная временная SQLite-БД, данные seed.py и случайный JWT-секрет. Основная БД не изменялась. Все приведённые ответы получены реальными HTTP-запросами curl.

Адрес при проверке: `http://127.0.0.1:50747`. В командах `BASE_URL` обозначает адрес сервера, `TOKEN`, токен первого запроса.

Одинаковые ошибки входа не доказывают защиту от определения аккаунта по времени ответа.

1. Успешный вход

```
curl -sS -X POST "$BASE_URL/auth/login" \
-H "Content-Type: application/json" \
--data '{"username": "alice", "password": "Alice-Str0ng-Pass!"}'
HTTP 200

{
  "access_token": "<JWT скрыт; полный токен использован в следующих запросах>",
  "expires_in": 3600,
  "token_type": "Bearer"
}
```

2. Неверный пароль

```
curl -sS -X POST "$BASE_URL/auth/login" \
-H "Content-Type: application/json" \
--data '{"username": "alice", "password": "wrong"}'
HTTP 401

{
  "error": "invalid credentials"
}
```

3. Неизвестный пользователь

```
curl -sS -X POST "$BASE_URL/auth/login" \
-H "Content-Type: application/json" \
--data '{"username": "nobody", "password": "wrong"}'
HTTP 401

{
  "error": "invalid credentials"
}
```

4. SQL-инъекция в логине

```
curl -sS -X POST "$BASE_URL/auth/login" \  
  -H "Content-Type: application/json" --data @- <<'JSON'  
{ "username": "' OR '1'='1' --", "password": "wrong"}  
JSON
```

HTTP 401

```
{  
  "error": "invalid credentials"  
}
```

5. Отсутствуют обязательные поля

```
curl -sS -X POST "$BASE_URL/auth/login" \  
  -H "Content-Type: application/json" \  
  --data '{}'
```

HTTP 400

```
{  
  "error": "username and password are required"  
}
```

6. Чтение без JWT

```
curl -sS -X GET "$BASE_URL/api/data"
```

HTTP 401

```
{  
  "error": "authorization header missing or malformed"  
}
```

7. Чтение с неверным JWT

```
curl -sS -X GET "$BASE_URL/api/data" \  
  -H "Authorization: Bearer invalid"
```

HTTP 401

```
{  
  "error": "invalid token"  
}
```

8. Чтение с действительным JWT

```
curl -sS -X GET "$BASE_URL/api/data" \  
  -H "Authorization: Bearer $TOKEN"
```

HTTP 200

```
{
  "count": 2,
  "items": [
    {
      "author": "admin",
      "body": "First post created by the seed script.",
      "created_at": "2026-09-30T10:50:26.266392",
      "id": 1,
      "title": "Welcome"
    },
    {
      "author": "alice",
      "body": "Access tokens are signed with HS256 and expire in one
hour.",
      "created_at": "2026-09-30T10:50:26.266880",
      "id": 2,
      "title": "Notes on JWT"
    }
  ]
}
```

9. Создание поста с XSS-строкой

```
curl -sS -X POST "$BASE_URL/api/posts" \
  -H "Content-Type: application/json" \
  --data '{"title": "XSS probe", "body": "<script>alert(1)</script>"}' \
  -H "Authorization: Bearer $TOKEN"
HTTP 201

{
  "author": "alice",
  "body": "&lt;script&gt;alert(1)&lt;/script&gt;",
  "created_at": "2026-09-30T10:50:26.745658+00:00",
  "id": 3,
  "title": "XSS probe"
}
```

10. Повторное чтение экранированного поста

```
curl -sS -X GET "$BASE_URL/api/data" \
  -H "Authorization: Bearer $TOKEN"
HTTP 200
```

```

{
  "count": 3,
  "items": [
    {
      "author": "admin",
      "body": "First post created by the seed script.",
      "created_at": "2026-09-30T10:50:26.266392",
      "id": 1,
      "title": "Welcome"
    },
    {
      "author": "alice",
      "body": "Access tokens are signed with HS256 and expire in one
hour.",
      "created_at": "2026-09-30T10:50:26.266880",
      "id": 2,
      "title": "Notes on JWT"
    },
    {
      "author": "alice",
      "body": "<script>alert(1)</script>",
      "created_at": "2026-09-30T10:50:26.745658",
      "id": 3,
      "title": "XSS probe"
    }
  ]
}

```

11. Создание поста без JWT

```

curl -sS -X POST "$BASE_URL/api/posts" \
  -H "Content-Type: application/json" \
  --data '{"title": "Denied", "body": "Denied"}'
HTTP 401

{
  "error": "authorization header missing or malformed"
}

```

ВЫВОД

Реализованы вход, чтение данных и создание постов. SQLAlchemy отделяет параметры от SQL, bcrypt хранит хэши паролей, middleware проверяет JWT, пользовательский текст экранируется при выдаче. Проверка зависимостей выявила уязвимую версию PyJWT; после обновления повторный аудит и тесты аутентификации прошли.

GitHub Actions успешно выполнил 51 тест, SAST и SCA. Подтверждённое ложное совпадение с PHP-библиотекой исключено адресно и объяснено в README. В отчёт включены ссылка на проверенный запуск, его скриншоты и фактические ответы curl.